

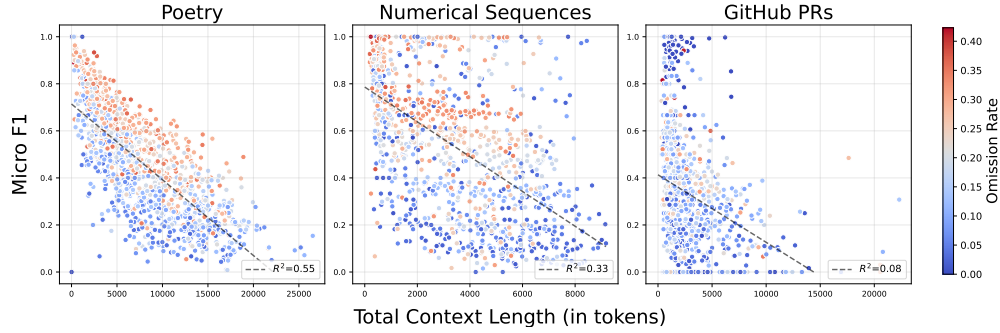


Figure 5: **GPT-4.1-mini performs worse on longer tasks in Poetry**, but the relationship is not clear in Numerical Sequences and Github PRs. Each plot shows the F1-score (y-axis) and the total context length (x-axis). Dark blue represents a lower and dark red represents a higher percentage of omission (number of omitted lines divided by total number of lines across each of three domains). Each dot on the graph represents the performance on a single instance. The dashed lines represent the least squares fit, with R^2 indicating the strength of correlation between the two axes.

might struggle to effectively handle "gaps" when processing documents containing omissions. We carry out additional experiments to assess this hypothesis in Section 4.2.

4 Analysis

We study the effect of perturbing the context length as well as the percentage of omissions on the models' performance. We additionally perform an error analysis and present a simple strategy that effectively addresses the difficulty in *AbsenceBench*. For the sake of cost, we limit our extended analysis to three LLMs: Claude-3.7-Sonnet, GPT-4.1-mini, and Llama-4-Maverick.

4.1 Perturbation Studies

We construct a perturbed dataset with a higher variance of omission rates in order to lower the impact of various factors on the difficulty of *AbsenceBench* tasks. We randomly sample the probability of omission to be $p \sim U(0, 0.5)$ for each document, and then randomly omit each element from that document with probability p . Figure 5 illustrates the relationship between the performance of GPT-4.1-mini and total context length. The analysis of Claude-3.7-Sonnet and Llama-4-Maverick are shown in Figure 6 and 7 in Appendix D.

Context length has a major impact on performance For contexts shorter than 5K tokens, the performance varies significantly with changes in context length. This finding contrasts with observations from the NIAH test, where context length affects performance only when contexts exceed 100K tokens. We perform a linear regression analysis to assess the correlation between F1-score and total context length. Results across three LLMs indicate a stronger average correlation ($R^2 = 0.55$) within the poetry domain compared to numerical sequences (0.19) and GitHub PRs (0.08).

LLMs perform worse with lower omission rate In Figure 5, we observe a notable pattern (as indicated by color) that shows a positive correlation between model's performance and the omission rate, especially in the poetry and numerical sequences domain. In other words, models actually perform *worse* when required to recall *fewer lines*. While this may initially appear counterintuitive, we hypothesize that a higher omission rate increases the likelihood of consecutive omissions occurring. Consequently, language models may be able generate text consistently until encountering a non-omitted element that disrupts their generation process. On the other hand, language models encounter greater difficulty with contexts containing fewer omissions, as numerous missing spans may interrupt the continuity between segments that must be generated.

Alternative prompt template reduces model performance Alongside the default prompt template (see Table 2), we curate a perturbed template where the task instructions are positioned after the

Models	Poetry		Numerical Sequences		GitHub PRs	
	<i>default</i>	<i>post-instruction</i>	<i>default</i>	<i>post-instruction</i>	<i>default</i>	<i>post-instruction</i>
Claude-3.7-Sonnet	78.8	66.1	94.3	93.7	35.1	34.7
GPT-4.1-mini	45.4	39.2	54.2	42.5	32.9	35.4
Llama-4-Maverick	48.7	36.6	71.5	63.6	29.6	30.5

Table 4: Micro F1-score (%) of three LLMs evaluated on the `AbsenceBench` with two different prompt templates. All three models exhibit reduced performance with the *post-instruction* template in the poetry and numerical sequences domains, but remain robust in the GitHub PRs domain.

Models	Poetry	Numerical Sequences	GitHub PRs	Average
Placeholder: None (baseline)				
Claude-3.7-Sonnet	73.5	91.4	35.7	66.9
GPT-4.1-mini	30.2	45.0	31.3	35.5
Llama-4-Maverick	32.8	58.7	29.0	40.2
Placeholder: <missing line>				
Claude-3.7-Sonnet	87.4 _(+18.9%)	97.7 _(+6.9%)	64.9 _(+81.8%)	83.3 _(+24.5%)
GPT-4.1-mini	50.4 _(+66.9%)	59.2 _(+31.5%)	46.8 _(+49.5%)	52.1 _(+46.8%)
Llama-4-Maverick	60.7 _(+85.0%)	78.9 _(+34.4%)	46.8 _(+61.4%)	62.1 _(+54.5%)
Placeholder: _____				
Claude-3.7-Sonnet	85.5 _(+16.3%)	97.2 _(+6.3%)	61.3 _(+71.7%)	81.3 _(+21.5%)
GPT-4.1-mini	45.9 _(+52.0)	57.3 _(+27.3%)	36.5 _(+16.6%)	46.5 _(+31.2)
Llama-4-Maverick	53.9 _(+64.3%)	73.0 _(+24.4)	36.5 _(+25.9%)	54.5 _(+35.5%)

Table 5: Micro F1-score of three LLMs evaluated on all three domains of `Absence Bench` using 2 different placeholders: “<missing line>” and “_____” (10 consecutive underlines), along with the percentage increase in F1-score compared to the none-placeholder baseline.

original and modified documents. We refer to this perturbed prompt template as “post-instruction”. Table 4 shows that the performance of three LLMs prompted with both prompt templates. All three models exhibit reduced performance with the post-instruction template under both the poetry and numerical sequences domains, suggesting that LLMs more effectively detect omissions when instructions are presented beforehand. Meanwhile, LLMs remain robust in the GitHub PRs domain. All detailed prompt templates are included in Appendix A.

4.2 Marking omissions with placeholders

To assess whether the Transformer’s attention mechanism is the reason for `AbsenceBench`’s difficulty, we explicitly mark omissions in the modified context using *placeholders*, special tokens that explicitly signal omitted segments. Specifically, we test across all domains (see Table 5 for full results) and consider two different placeholders: “<missing line>” and 10 consecutive underlines. Across all three domains, using “<missing line>” as the placeholder performs better, with an average boost in performance of 41.9%. Notably, Llama-4-Maverick achieves the highest improvement of 54.5% overall. The improved performance is comparable to that of o3-mini, yet o3-mini generates 9.1K extra thinking tokens on average.

Why does having a placeholder help so much? We hypothesize that `AbsenceBench` is so difficult for models because, when a segment of text is omitted, there is no position to attend to that corresponds directly to that omission. Transformer-based LLMs are incredibly good at finding the correct piece of information to attend to from a document, but what happens when the information is a conspicuous absence? What should the Transformer attend to? The fact that including a placeholder improves performance so drastically suggests that Transformer self-attention struggles to anchor on information gaps. This is a key area for future architecture and inference-time research, as omissions are common—from missing paperwork to comments deliberately left out of a recommendation letter.

5 Discussion

Our experiments highlight a failure of cutting-edge models to consistently succeed at the simple task of detecting absences, despite very strong performance on existing benchmarks. This suggests that popular evaluations do not effectively cover the capability of absence recognition, although it is fundamental for applications such as LLM-as-a-judge in which models must recognize which rubric elements are not covered. Our results are especially surprising given the simplicity of our evaluation: we only ask about surface-form absence (rather than absence on a higher semantic level), and directly provide models with both the original and modified documents for correct comparison. Indeed, our tasks could be solved by a simple program in linear time, yet deployed LLMs with hundreds of billions of parameters consistently struggle on them.

We propose an initial hypothesis explaining this behavior: identifying presence is simpler than absence with the attention mechanisms underlying Transformers (Vaswani et al., 2017). Information included in a document can be directly attended to, while the absence of information cannot. We provide initial evidence of this hypothesis in §4.2. Rather than simply deleting elements from the original document, we include an absence “placeholder” here (e.g. “<missing line>”) to provide a span of text for the transformer to attend to, indicating absence. This causes consistent, significant improvements for models, suggesting that the lack of a sequence to attend to is at least one aspect of this problem.

One important question raised by our work is how to make models more effective at handling and identifying absence. The strong performance of Gemini-2.5-flash in 2 of our 3 domains suggests that inference-time compute and reasoning mechanisms may help, although this may not hold in general. However there is a cost: we find that Gemini uses an extremely large number of reasoning tokens in many cases (Figure 3), in many cases exceeding the length of the original document by an order of magnitude. This could be the result of naive solutions, such as regenerating the full document multiple times. While this would work for the simple versions of absence studied here, it would likely fail on more complex notions of absence that look beyond missing surface forms.

We suggest a few directions where future work can go. Reasoning may improve absence detection, but must be studied on more complex, *semantic* notions of absence to be sufficiently validated—AbsenceBench is merely a necessary bar. New architectures, with mechanisms significantly different from attention, may be required to address the deeper problem. Such work should be supported by a more mechanistic understanding of why existing models often fail on these tasks, and how attention is connected to this question. We hope that AbsenceBench will provide a natural playground for such mechanistic studies. This will also be key in guiding how existing models are used—if they fail at these simple notions of absence, it is not clear that models will be trustworthy for more complex tasks that require this capability, such as model-as-a-judge evaluations using rubrics.

Regardless of how this problem might be solved, our work supports the notion that LLMs show *novel* intelligence, not well explained by analogy to humans. LLMs fail at this simple notion of absence while performing at superhuman levels on tasks that are quite similar (e.g. Needle-in-a-haystack) and many tasks that seem much more complex and challenging (e.g. math problem solving). The “shape” of LLM intelligence is simply not that similar to humans. While improving LLM performance on absence should be one goal, this also poses an opportunity to consider: how might we map the axes of variation in LLM intelligence?

Our findings suggest that models are not effective at understanding the conspicuous absence of information, an often overlooked yet foundational component of comprehension. Standard benchmarks overwhelmingly focus on whether models can identify and reason about what is present, which does not appear to tightly correlate with a model’s ability to identify absent information. However use-cases such as LLM-as-a-judge, and tasks such as grading, legal reasoning, and misinformation detection, often hinge on recognizing what is missing. The gap in performance between NIAH and AbsenceBench underscores how misleading current evaluations might be if they ignore absence. Evaluators and developers should thus augment rubric-based assessments with systematic tests for absence sensitivity, both at surface and semantic levels. More broadly, absence may be a useful conceptual lens through which to understand model failure. Rather than treating hallucinations, misjudgments, or oversights as separate phenomena, they may all be related to a common limitation: models’ weak grasp of what is not there. As a result, better understanding and diagnosing absence failure may reveal general-purpose principles for more robust and trustworthy LLM behavior.